Topic 11: Collective Communications

Contents

- layout.csl
- pe_program.csl
- run.py
- commands.sh

The `<collectives_2d>` library can be used for communication between PEs in the same row or column. It mimics the capabilities provided by message passing interface (MPI) collective operations found in other programming languages.

This example showcases each of the currently available communication primitives while using the library across two indepedent dimensions. The communication tasks are executed asynchronously.

`task_x` uses the `broadcast` primitive to transmit data from the first PE in every row to every other PE in the same row. After the data is received, `reduce_fadds` computes the vector sum of the `broadcast_recv`. The result is transmitted back to the first PE in every row.

`task_y` operates concurrently along every column of PEs. The task first uses `scatter` to distribute `chunk_size` slices of `scatter_data` across every other PE in the same column. The task uses `gather` to collect `chunk_size` slices of data distributed by `scatter`. Because `scatter` is the inversion of `gather`, we have used collective communications to transmit the data from `scatter_data` to `gather_recv`.

layout.csl

```
// color/ task ID map
//
//  ID var              ID var             ID var                ID var
//   0 c2d_x_color_0     9                 18                    27 reserved (memcpy)
//   1 c2d_x_color_1    10 c2d_x_entrypt_0 19                    28 reserved (memcpy)
//   2                  11 c2d_x_entrypt_1 20                    29 reserved
//   3                  12 c2d_y_entrypt_0 21 reserved (memcpy)  30 reserved (memcpy)
//   4 c2d_y_color_0    13 c2d_y_entrypt_1 22 reserved (memcpy)  31 reserved
//   5 c2d_y_color_1    14                 23 reserved (memcpy)  32
//   6                  15 task_x_id       24                    33
//   7                  16 task_y_id       25                    34
//   8                  17                 26                    35
//

param Pw:         u16; // kernel width
```

```
param Ph:         u16; // kernel height
param chunk_size: u16; // Num elements to send/recv in collectives

// Colors
const c2d_x_color_0: color = @get_color(0);
const c2d_x_color_1: color = @get_color(1);
const c2d_y_color_0: color = @get_color(4);
const c2d_y_color_1: color = @get_color(5);

// Task IDs
const c2d_x_entrypt_0: local_task_id = @get_local_task_id(10);
const c2d_x_entrypt_1: local_task_id = @get_local_task_id(11);
const c2d_y_entrypt_0: local_task_id = @get_local_task_id(12);
const c2d_y_entrypt_1: local_task_id = @get_local_task_id(13);
const task_x_id:       local_task_id = @get_local_task_id(15);
const task_y_id:       local_task_id = @get_local_task_id(16);

const c2d = @import_module("<collectives_2d/params>");

const memcpy = @import_module( "<memcpy/get_params>", .{
  .width = Pw,
  .height = Ph
});

layout {
  @set_rectangle(Pw, Ph);

  var Px: u16 = 0;
  while (Px < Pw) : (Px += 1) {
    var Py: u16 = 0;
    while (Py < Ph) : (Py += 1) {
      const params = c2d.get_params(Px, Py, .{
        .x_colors      = .{ c2d_x_color_0,   c2d_x_color_1 },
        .x_entrypoints = .{ c2d_x_entrypt_0, c2d_x_entrypt_1 },
        .y_colors      = .{ c2d_y_color_0,   c2d_y_color_1 },
        .y_entrypoints = .{ c2d_y_entrypt_0, c2d_y_entrypt_1 },
      });
      const memcpy_params = memcpy.get_params(Px);
      @set_tile_code(Px, Py, "pe_program.csl", .{
        .memcpy_params = memcpy_params,
        .c2d_params = params,
        .chunk_size = chunk_size,
        .task_x_id = task_x_id,
        .task_y_id = task_y_id });
    }
  }

  // export symbol name
  @export_name("broadcast_data", [*]u32, true);
  @export_name("scatter_data", [*]u32, true);
  @export_name("broadcast_recv", [*]u32, true);
  @export_name("faddh_result", [*]u32, true);
  @export_name("gather_recv", [*]u32, true);

  @export_name("f_run_x", fn()void);
```

```
  @export_name("f_run_y", fn()void);
}
```

pe_program.csl

```
param c2d_params;
param memcpy_params;

param chunk_size: u16; // Number of elements to send/recv in collectives

// Task IDs
param task_x_id: local_task_id; // Task ID for callback for collectives in x direction
param task_y_id: local_task_id; // Task ID for callback for collectives in y direction

const sys_mod = @import_module( "<memcpy/memcpy>", memcpy_params);

const rect_height = @get_rectangle().height;
const rect_width = @get_rectangle().width;

const mpi_x = @import_module("<collectives_2d/pe>", .{
    .dim_params = c2d_params.x,
    .queues = [2]u16{2,4},
    .dest_dsr_ids = [1]u16{1},
    .src0_dsr_ids = [1]u16{1},
    .src1_dsr_ids = [1]u16{1}
    });
const mpi_y = @import_module("<collectives_2d/pe>", .{
    .dim_params = c2d_params.y,
    .queues = [2]u16{3,5},
    .dest_dsr_ids = [1]u16{2},
    .src0_dsr_ids = [1]u16{2},
    .src1_dsr_ids = [1]u16{2}
    });


const Nx = chunk_size * rect_width;
const Ny = chunk_size * rect_height;

// broadcast_data and scatter_data supplied by run.py
var broadcast_data = @zeros([Nx]u32);
var broadcast_recv = @zeros([Nx]u32);
var faddh_result = @zeros([Nx]u32);

var scatter_data = @zeros([Ny]u32);
var scatter_recv = @zeros([Ny]u32);
var gather_recv = @zeros([Ny]u32);

var ptr_broadcast_data: [*]u32 = &broadcast_data;
var ptr_scatter_data: [*]u32 = &scatter_data;
var ptr_broadcast_recv: [*]u32 = &broadcast_recv;
var ptr_faddh_result: [*]u32 = &faddh_result;
var ptr_gather_recv: [*]u32 = &gather_recv;
```

```
var task_x_state: u16 = 0;
task task_x() void {
   switch (task_x_state) {
      0 => {
         mpi_x.init();
         var send_buf = @ptrcast([*]u32, &broadcast_data);
         var recv_buf = @ptrcast([*]u32, &broadcast_recv);
         if (mpi_x.pe_id == 0) {
            mpi_x.broadcast(0, send_buf, Nx, task_x_id);
         } else {
            mpi_x.broadcast(0, recv_buf, Nx, task_x_id);
         }

         task_x_state += 1;
      },
      1 => {
         var send_buf = @ptrcast([*]f32, &broadcast_recv);
         var recv_buf = @ptrcast([*]f32, &faddh_result);

         mpi_x.reduce_fadds(0, send_buf, recv_buf, Nx, task_x_id);

         task_x_state += 1;
      },
      else => {
         // WARNING: the user must unblock cmd color for every PE
         sys_mod.unblock_cmd_stream();
         return;
      }
   }
}

var task_y_state: u16 = 0;
task task_y() void {
   switch (task_y_state) {
      0 => {
         mpi_y.init();
         var send_buf = @ptrcast([*]u32, &scatter_data);
         var recv_buf = @ptrcast([*]u32, &scatter_recv);

         mpi_y.scatter(0, send_buf, recv_buf, chunk_size, task_y_id);

         task_y_state += 1;
      },
      1 => {
         var send_buf = @ptrcast([*]u32, &scatter_recv);
         var recv_buf = @ptrcast([*]u32, &gather_recv);

         mpi_y.gather(0, send_buf, recv_buf, chunk_size, task_y_id);

         task_y_state += 1;
      },
      else => {
         // WARNING: the user must unblock cmd color for every PE
         sys_mod.unblock_cmd_stream();
```

```
            return;
        }
    }
}

comptime {
    @bind_local_task(task_x, task_x_id);
    @bind_local_task(task_y, task_y_id);
}

fn f_run_x() void {
    @activate(task_x_id);

    // terminate when task_x finishes
}

fn f_run_y() void {
    @activate(task_y_id);

    // terminate when task_y finishes
}

comptime{
  @export_symbol(ptr_broadcast_data, "broadcast_data");
  @export_symbol(ptr_scatter_data, "scatter_data");
  @export_symbol(ptr_broadcast_recv, "broadcast_recv");
  @export_symbol(ptr_faddh_result, "faddh_result");
  @export_symbol(ptr_gather_recv, "gather_recv");
  @export_symbol(f_run_x);
  @export_symbol(f_run_y);
}
```

run.py

```
#!/usr/bin/env cs_python

import argparse
import json
import numpy as np

from cerebras.sdk.runtime.sdkruntimepybind import SdkRuntime, MemcpyDataType # pylint:
disable=no-name-in-module
from cerebras.sdk.runtime.sdkruntimepybind import MemcpyOrder # pylint: disable=no-name-in-
module

parser = argparse.ArgumentParser()
parser.add_argument('--name', help='the test name')
parser.add_argument('--cmaddr', help='IP:port for CS system')
args = parser.parse_args()
dirname = args.name

# Parse the compile metadata
```

```python
with open(f"{dirname}/out.json", encoding="utf-8") as json_file:
  compile_data = json.load(json_file)
params = compile_data["params"]
Pw = int(params["Pw"])
Ph = int(params["Ph"])
chunk_size = int(params["chunk_size"])
print(f"Pw = width of the core = {Pw}")
print(f"Ph = height of the core = {Ph}")
print(f"chunk_size = {chunk_size}")

Nx = Pw*chunk_size
Ny = Ph*chunk_size

print(f"Nx = {Nx}, Ny = {Ny}")

memcpy_dtype = MemcpyDataType.MEMCPY_32BIT
runner = SdkRuntime(dirname, cmaddr=args.cmaddr)

sym_broadcast_data = runner.get_id("broadcast_data")
sym_scatter_data = runner.get_id("scatter_data")
sym_broadcast_recv = runner.get_id("broadcast_recv")
sym_faddh_result = runner.get_id("faddh_result")
sym_gather_recv = runner.get_id("gather_recv")

runner.load()
runner.run()

print("step 1: copy mode H2D(broadcast_data) to 1st column PEs")
broadcast_data = np.ones((Ph, 1, Nx)).astype(np.float32)
runner.memcpy_h2d(sym_broadcast_data, broadcast_data.ravel(), 0, 0, 1, Ph, Nx, \
    streaming=False, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=True)

print("step 2: copy mode H2D(scatter_data) to 1st row PEs")
scatter_data = np.ones((1, Pw, Ny)).astype(np.int32)
runner.memcpy_h2d(sym_scatter_data, scatter_data.ravel(), 0, 0, Pw, 1, Ny, \
    streaming=False, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=True)

print("step 3: call f_run_x to test broadcast and reduction")
runner.launch("f_run_x", nonblock=False)

print("step 4: call f_run_y to test scatter and gather")
runner.launch("f_run_y", nonblock=False)

print("step 5: copy mode D2H(broadcast_recv)")
# broadcast on x: Px=0 broadcasts data to all other PEs
# broadcast_recv(y, x=0) = 0
# broadcast_recv(y, x !=0) = ones
broadcast_recv_1d = np.zeros(Ph*Pw*Nx, np.float32)
runner.memcpy_d2h(broadcast_recv_1d, sym_broadcast_recv, 0, 0, Pw, Ph, Nx, \
    streaming=False, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=False)
broadcast_recv = broadcast_recv_1d.reshape((Ph, Pw, Nx))

print("step 6: copy mode D2H(faddh_result) from 1st column PEs")
# reduce(broadcast_recv) to Px=0
faddh_result_1d = np.zeros(Ph*Nx, np.float32)
```

```
runner.memcpy_d2h(faddh_result_1d, sym_faddh_result, 0, 0, 1, Ph, Nx, \
    streaming=False, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=False)
faddh_result = faddh_result_1d.reshape((Ph, 1, Nx))

print("step 7: copy mode D2H(gather_recv) from 1st row PEs")
gather_recv_1d = np.zeros(Pw*Ny, np.int32)
runner.memcpy_d2h(gather_recv_1d, sym_gather_recv, 0, 0, Pw, 1, Ny, \
    streaming=False, data_type=memcpy_dtype, order=MemcpyOrder.ROW_MAJOR, nonblock=False)
gather_recv = gather_recv_1d.reshape((1, Pw, Ny))

runner.stop()

# verify broadcast on x-direction
correct_broadcast_recv = np.ones(Nx).astype(np.float32)
for y in range(Ph):
  for x in range(Pw):
    if x == 0:
      continue
    np.testing.assert_equal(broadcast_recv[y, x], correct_broadcast_recv)

# verify faddh_result at 1st column PEs
# reduce on x: reduce(broadcast_recvs) to Px=0
# where broadcast_recvs(y, x=0) = 0
#       broadcast_recvs(y, x != 0) = ones
correct_faddh_result = np.full(Nx, (Pw-1), dtype=np.float32)
for y in range(Ph):
  np.testing.assert_equal(faddh_result[y, 0], correct_faddh_result)

# verify gather_recv at 1st row PEs
correct_gather_recv = np.ones(Ny).astype(np.int32)
for x in range(Pw):
  np.testing.assert_equal(gather_recv[0, x], correct_gather_recv)

print("SUCCESS")
```

commands.sh

```
#!/usr/bin/env bash

set -e

cslc --arch=wse3 ./layout.csl --fabric-dims=22,17 --fabric-offsets=4,1 \
--params=Pw:15,Ph:15,chunk_size:3 -o out \
--memcpy --channels=1 --width-west-buf=0 --width-east-buf=0
cs_python run.py --name out
```


Last updated on Apr 15, 2026, 7:13:14 PM.